MINESWEEPER

GAME ON FPGA

Project Documentation

Course: Digital Systems Design / VHDL / FPGA Project
Project topic: VGA Minesweeper Game using ROM, RAM, PS/2 Mouse and Display Logic

Autor: Friciu Mara - Alexandra

CONTENTS

1. CHAPTER 1 : PROJECT SPECIFICATIONS

You are part of a team tasked with designing a digital implementation of the classic Minesweeper game, intended to run on an FPGA board using VHDL. The system displays the game on a VGA monitor and allows user interaction through a PS/2 mouse and FPGA switches.

The design uses a combination of ROM memories, RAM memory, and control logic to simulate the behavior of the Minesweeper game.

Game Description

The game consists of a grid (e.g., 8×8), where each cell may contain:

- a bomb
- an empty space
- a number (indicating adjacent bombs)

The goal of the player is to reveal all non-bomb cells without selecting a bomb.

The player interacts with the game by selecting individual cells on the grid. A cell can be opened to reveal its content.

When a cell is opened, its content is displayed. If the cell contains a number, that number represents the total count of bombs located in the eight neighboring cells surrounding it, including diagonal positions.

If the opened cell contains a bomb, the game is immediately lost.

If the opened cell is empty, the system automatically reveals adjacent empty cells, resulting in a chain expansion that uncovers a larger portion of the grid.

Operating Procedure

● Initial State

- All cells are **hidden (closed)**
- RAM is initialized to: 00 = closed
- The VGA display shows a fully covered board

● User Interaction

The user interacts with the system using switches to select the level and to restart the game. Moreover, the user uses the mouse to choose a cell.

- **Right Click** → reveals what is under a closed cell

● Memory Behavior

Board ROM (Game Map)

The Board ROM is used to store the fixed content of the Minesweeper board. Since the board does not change during execution, the ROM behaves as a read-only memory.

Each memory address corresponds to one cell from the board. For an 8 x 8 board, the ROM contains 64 positions. The address is generated from the current cell coordinates.

- Stores the actual content of the board:
  - ➢ 0000 = empty
  - ➢ 0001 = 1
  - ➢ 0010 = 2
  - ➢ 0011 = 3
  - ➢ 0100 = 4
  - ➢ 0101 = 5
  - ➢ 0110 = 6
  - ➢ 0111 = 7
  - ➢ 1000 = 8
  - ➢ 1001 = bomb
  - ➢ 1010 = empty non-opened

RAM (Player View)

The RAM is used to store the current state of the board during the game. Unlike the ROM, the RAM can be modified while the project is running.

The RAM stores information about the visible state of each cell, for example whether the cell is opened or still hidden.

Initial visible value = "1010" -- empty non-opened / hidden cell

When the player opens a cell, the project writes into RAM the real value read from the Board ROM at the same address. After this write operation, the display logic will show the opened value instead of the hidden cell.

RAM[address] <= Board_ROM[address]

Signals:

- ☐ **Address:** selects the current board cell
- ☐ **Data_in:** value that will be written into RAM
- ☐ **Data_out:** visible value currently read from RAM
- ☐ **WE:** write enable signal
- ☐ **Clk:** clock used for the write operation

Display Logic

The display logic is responsible for drawing the correct image for every visible cell on the VGA screen. The VGA pixel coordinates are converted into board coordinates and local coordinates inside the selected cell.

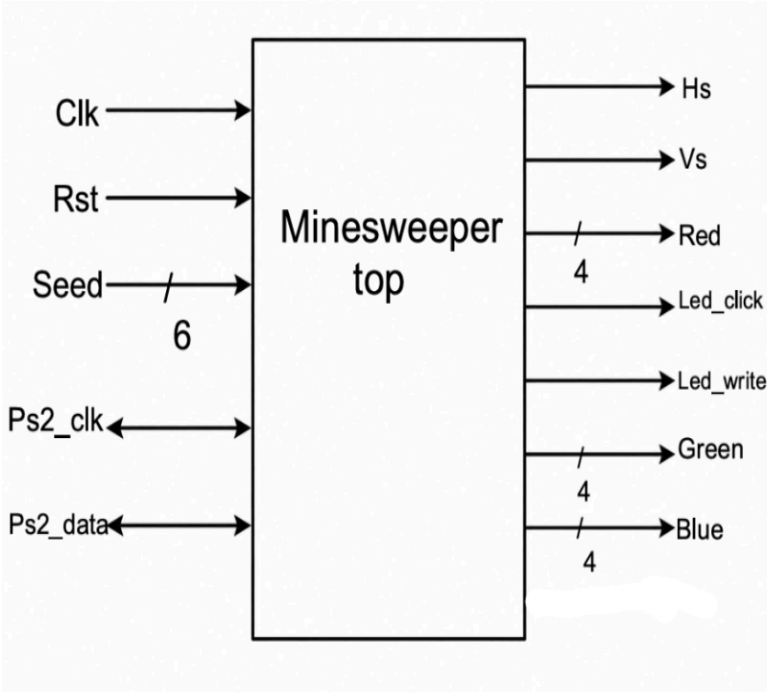
1. Check if the current VGA pixel is inside the Minesweeper board area.
2. Calculate cell_x and cell_y from the VGA coordinates.
3. Generate the cell address from cell_x and cell_y.
4. Read the visible cell value from RAM.
5. Send the RAM output as sel_code to the demultiplexer.
6. Activate the correct display case or sprite ROM.
7. Output the final VGA color for the current pixel. If the current pixel is outside the board area, the display logic should draw the background color instead of a cell image

Display Cases

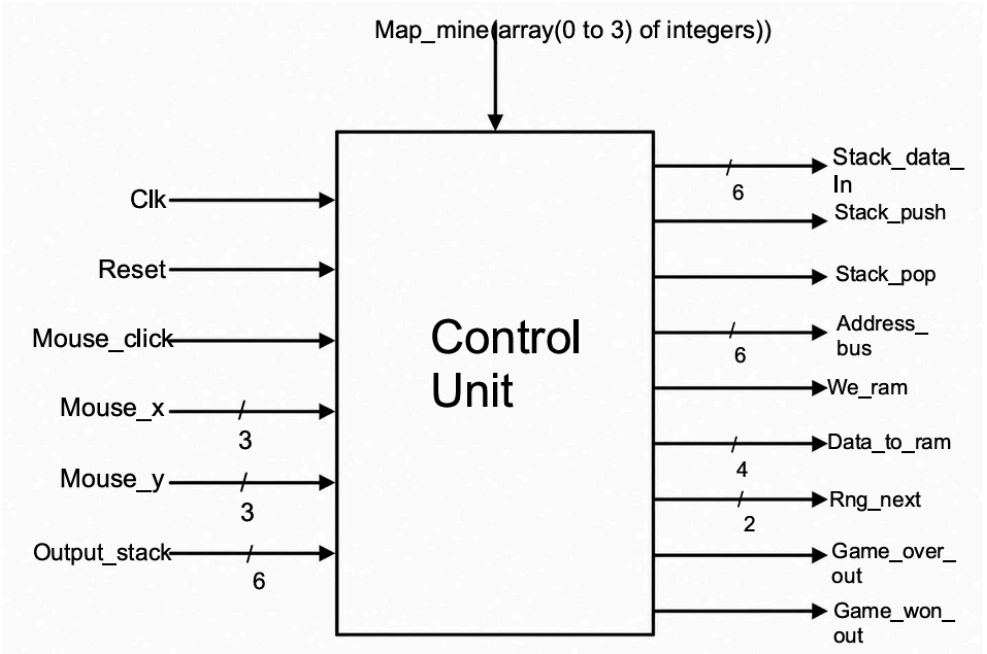
- If the cell is marked as *closed* in RAM, the system displays the closed cell graphical element, regardless of the underlying content
- If the cell is marked as *opened* in RAM, the system retrieves the corresponding value from the Board ROM and displays the appropriate graphical element:
  - Empty cell → display empty tile
  - Numbered cell → display the corresponding number from 1 to 8
  - Bomb cell → display bomb tile

2. CHAPTER 2 : DESIGN

2.1 Black Box



2.2 Control and Execution Unit



The Control Unit acts as the central brain of the system. It processes control signals from input peripherals and coordinates the generation of memory addresses and write/read signals for the Player View RAM and other internal modules.

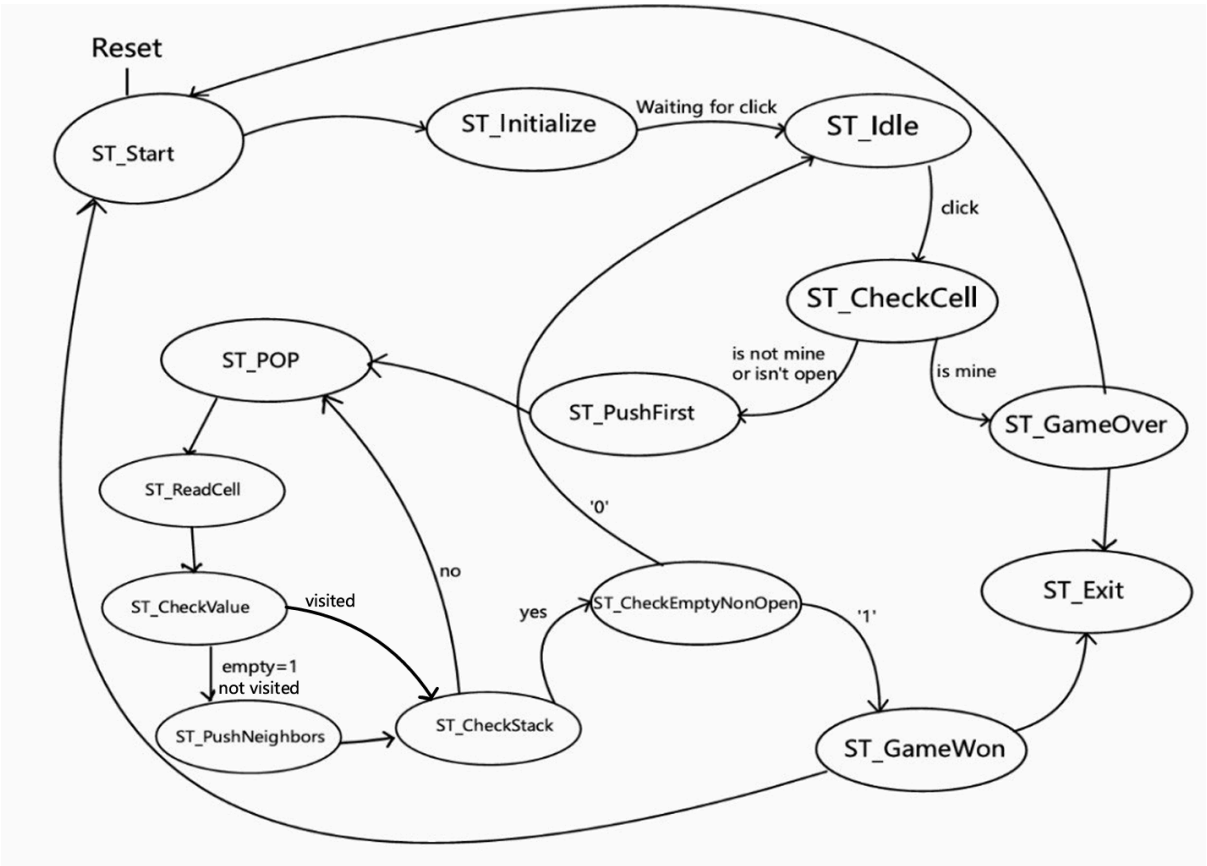
INPUT PORTS:

- Clk: The global system clock signal
- Reset: System-wide reset to initialize or restart the game
- Mouse_click: Triggered when the user presses the left mouse button
- Mouse_x(3 bits)/Mouse_y(3 bits): The specific row and column coordinates of the selected cell within the 8 x 8 grid
- Output_stack(6 bits): Data read from the stack memory during the automatic flood-fill expansion
- Map_mine: An array of integers containing the actual layout and location of the hidden bombs

OUTPUT PORTS:

- Stack_data_In (6 bits) / Stack_push / Stack_pop: The hardware control interface utilized to interact with the stack module
- Address_bus (6 bits): Main address bus allocated for indexing the 64 unique locations (0 to 63) of the RAM
- We_ram (Write Enable): Active-high signal enabling write operations to update a cell's state in RAM
- Data_to_ram (4 bits): The updated state value being committed to the player view RAM
- Rng_next: Control signal used to advance the pseudo-random number generator to the next value.
- Game_over_out / Game_won_out: Status signals routed directly to the display logic to indicate terminal game conditions

2.3. State diagram of Control Unit

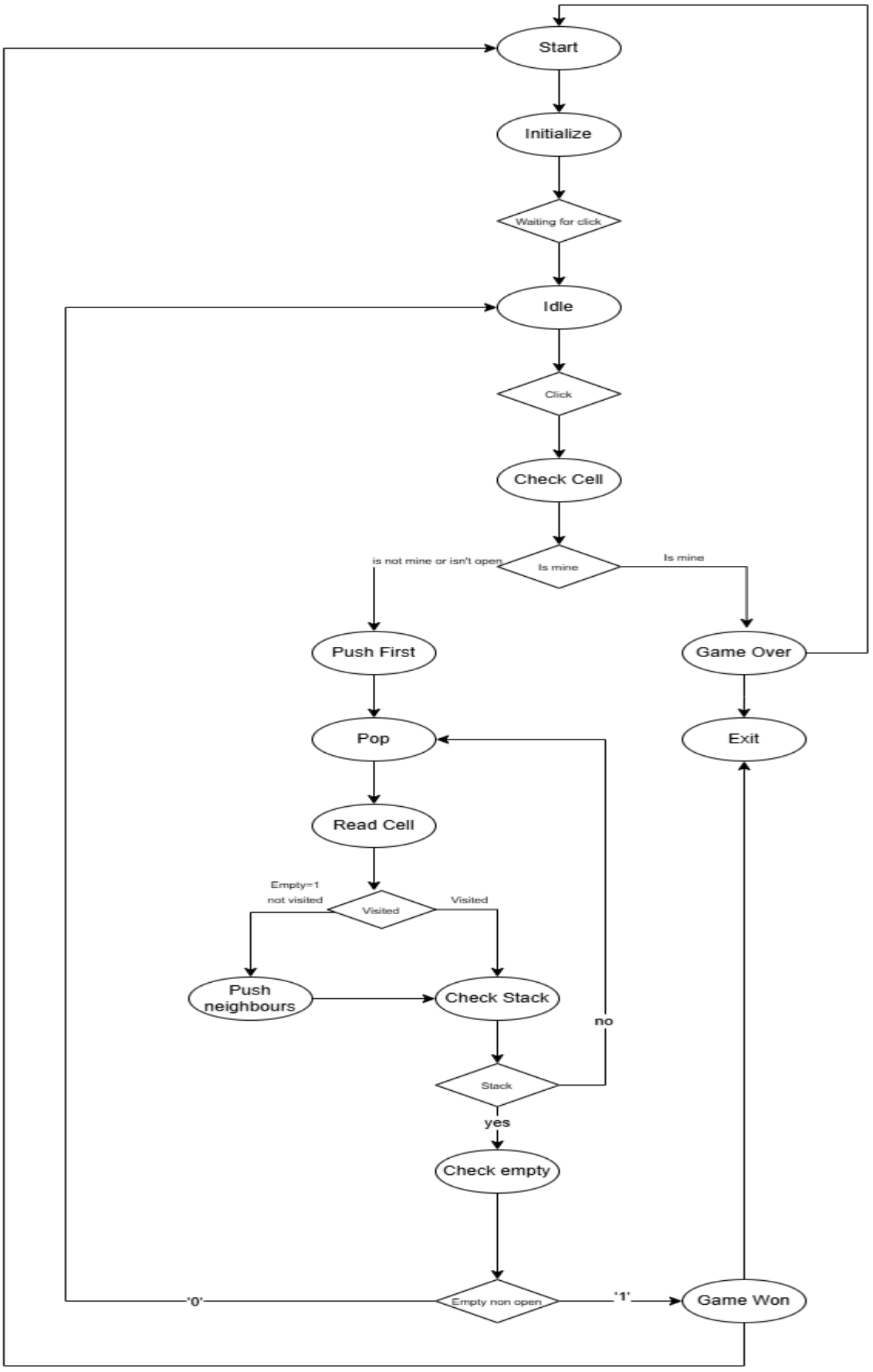


The operational workflow of the game logic is governed by a synchronous Finite State Machine (FSM). This state machine handles user inputs and manages the stack-driven iterative loop required for the chain expansion of empty cells:

1. **ST_Start:** The initial entry point triggered upon a hardware power-on or when the **Reset** signal is pulled high.
2. **ST_Initialize:** Loops through the RAM to format all 64 cells to the hidden/closed state (**00**) and resets the bomb generator.
3. **ST_Idle:** The system enters a low-power listening mode (**Waiting for click**). It continuously polls for a valid user interaction. When **Mouse_click** is detected, the FSM transitions to **ST_CheckCell**.
4. **ST_CheckCell:** The system interrogates the Board ROM map at the clicked coordinates:
    - If the target cell contains a bomb (**is mine**), the FSM branches straight to **ST_GameOver**.
    - If the cell is safe, it transitions to **ST_PushFirst** to begin revealing the board.
5. **ST_PushFirst:** The 6-bit index of the clicked cell is pushed into the stack module (**Stack_push**) to seed the reveal algorithm.

6. **ST_POP:** Pulls a single cell address out of the stack structure to process its immediate neighborhood.
7. **ST_ReadCell:** Fetches the underlying value from the Board ROM and checks its current visibility flag within the Player View RAM.
8. **ST_CheckValue:** Evaluates the cell properties:
    - If the cell is blank/empty (**empty=1**) and has not been opened yet, it moves to **ST_PushNeighbors**.
    - If it has already been visited or reveals an explicit bomb-count number, it skips neighborhood processing and moves to **ST_CheckStack**.
9. **ST_PushNeighbors:** Generates the indices for all 8 surrounding neighbor cells and pushes them onto the hardware stack for subsequent scanning.
10. **ST_CheckStack:** Inspects the status flags of the stack component:
    - If the stack is not empty (**no**), the loop re-enters **ST_POP** to consume the next queued address.
    - If the stack is empty (**yes**), the continuous cascade expansion is complete, and it advances to **ST_CheckEmptyNonOpen**.
11. **ST_CheckEmptyNonOpen:** Analyzes whether any unrevealed non-bomb cells remain on the grid:
    - If unrevealed safe cells persist (0), the game continues normally and resets back to **ST_Idle**.
    - If all safe cells are successfully opened (1), the FSM branches directly to **ST_GameWon**.
12. **ST_GameOver / ST_GameWon:** Terminal states that halt standard game loop processing and assert the respective outcome flags. Exiting these states passes through **ST_Exit** to prepare the parameters for a fresh instance.

2.4 Detailed diagram



3. CHAPTER 3 : USER MANUAL

The Minesweeper implementation provides an interactive gaming experience deployed directly onto an FPGA development workspace. The system utilizes physical switches to initialize the board configuration and a PS/2 mouse for gameplay.

3.1 Hardware Interfacing & Configuration

To launch and play the game, set up the following physical connections and hardware controls on the FPGA board:

1. **Display Interface:** Hook up a standard monitor to the native VGA port using a high-density D-sub 15-pin VGA cable.
2. **Input Device:** Connect a legacy PS/2 mouse (or an appropriate USB-to-PS/2 protocol adapter mouse) into the integrated PS/2 peripheral port.
3. **Power & Programming:** Power up the FPGA module and use the development suite to flash the compiled VHDL bitstream onto the onboard chip.

3.2 Game Initialization Setup

Before beginning a match, the player must seed the game board to generate a pseudo-random layout of mines. Follow this step-by-step procedure using the physical onboard switches:

1. **Seed Selection (Switches 1 to 6):** Toggle the onboard switches from SW[1] to SW[6] into a custom configuration of high (1) and low (0) positions. This binary combination acts as the initial "seed" or level profile for the Pseudo-Random Number Generator (PRNG). Changing this combination alters the distribution and pattern of the mines for that session.
2. **Reset Activation:** Once your chosen seed combination is set on switches 1 to 6, toggle the dedicated **Reset Switch** to its active high position → reset on switch 0.
3. **VGA Initialization:** Activating the reset switch triggers the finite state machine (ST_Initialize), which reads the selected seed, shuffles the hidden bomb map, and populates the volatile RAM. The connected VGA monitor will immediately update, displaying a completely blank 8 x 8 grid of uniform covered gray squares.
4. **Start Game:** Toggle the Reset Switch back to its inactive position to release the system into its standby listening loop (ST_Idle), allowing it to register mouse inputs.
5. **Mouse:** The project uses the board's USB HID connector in PS/2-compatible mode. In the constraints file, the mouse clock and data lines are connected to the following FPGA pins:

ps2_clk → C17 → Clock generated by the mouse during PS/2 communication

ps2_data → B17 → Serial data sent by the mouse

3.3 Game Rules & Controls

Once initialized, the board is locked into play mode, and all interaction shifts to the mouse device:

1. **Targeting:** Move the connected mouse across your desktop. An on-screen hardware cursor tracking the movement allows you to point at individual squares within the 8x8 grid.

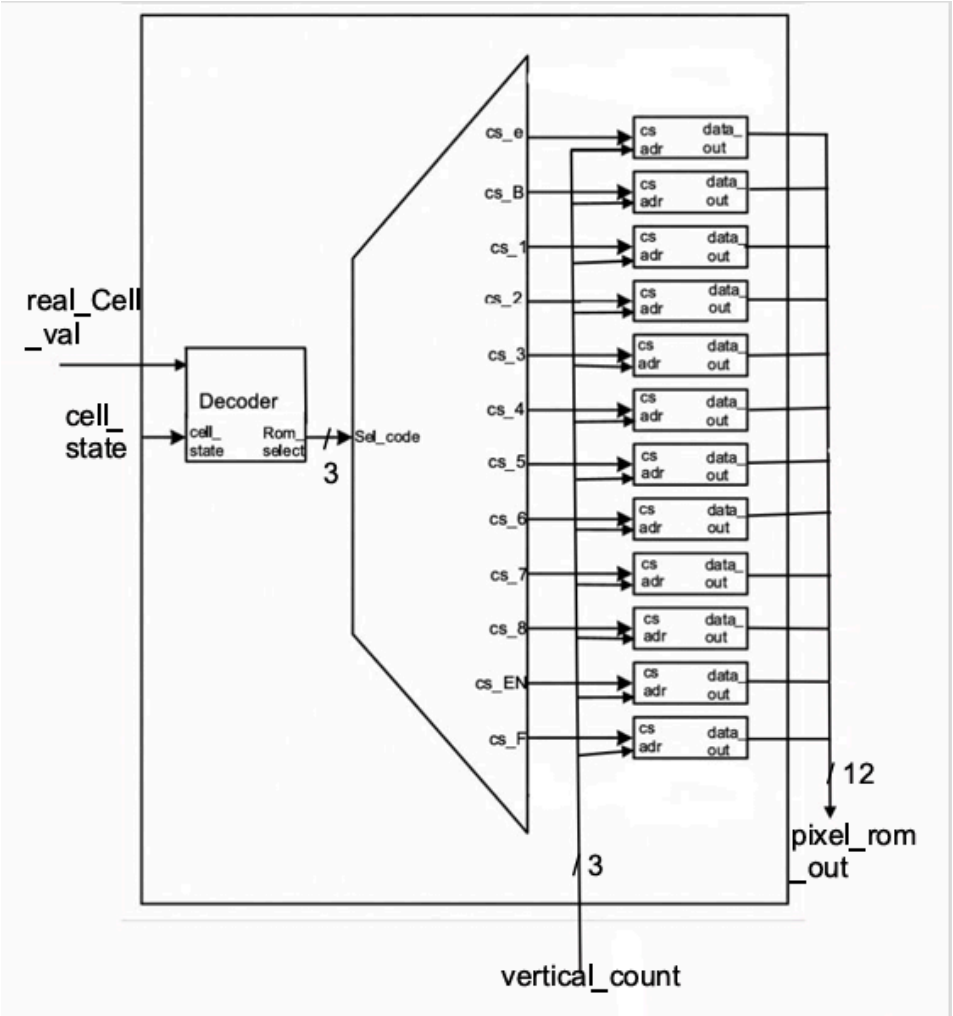
2. **Revealing Squares:** Align the pointer over a target grid square and press the **right click on the mouse** to discover its contents.
    - Revealing a numbered square displays a monochrome digit indicating the local bomb density.
    - Revealing an empty region triggers an immediate automated cascade that safely clears out the adjacent blank sector.

3. **End Conditions:**
    - **Game Over:** If you click on a hidden mine, the grid reveals the bomb and changes the board outline to red .
    - **Victory:** Win the game by clearing away all non-bomb tiles, leaving only the squares containing hidden mines unselected. The board outline changes to green.

4. CHAPTER 4 : DESIGN SOLUTIONS JUSTIFICATION

VGA

To maintain a stable, real-time video signal without screen flickering or exceeding the available memory resources of the FPGA, a **pixel-accurate indirect decoding layout** was implemented.



The VGA module is responsible for generating the video signal required to display the game on a monitor. It continuously produces pixel coordinates (pixel_x, pixel_y), which determine the current position on the screen.

These coordinates are used to identify the corresponding cell in the game grid. Based on this, the system computes a memory address that is used to access the player state stored in RAM.

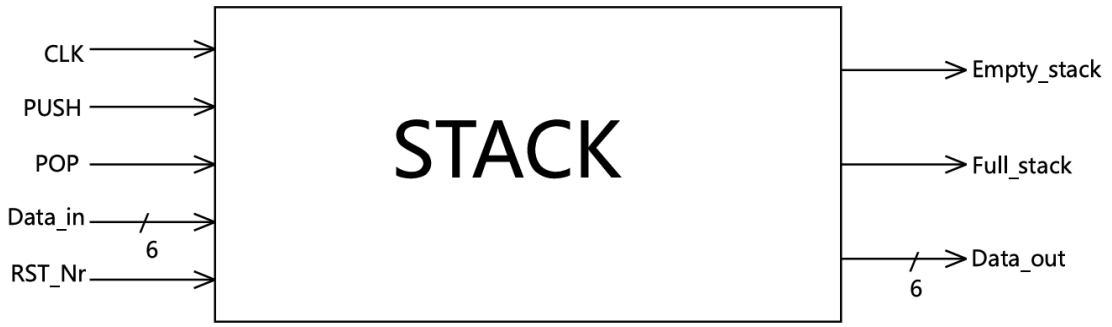
A decoding unit combines the player state with the real cell content and generates a selection signal that activates one of the graphical ROM memories. Each ROM contains the image of a specific cell type (e.g., closed, empty, numbered, or bomb).

The VGA grid is generated by continuously scanning the screen with horizontal and vertical pixel counters.

First, the circuit checks if the current pixel is inside the visible 640×480 area and then verifies if it belongs to the 8×8 Minesweeper board. If the pixel is inside the board, its position is converted into cell coordinates and local sprite coordinates, so the correct cell image can be displayed.

The color process then draws the black grid lines, the gray cell background, and the sprite pixels according to **pixel_bit**. The outer border is also controlled by the game state, becoming gray during gameplay, red on game over, and green when the player wins.

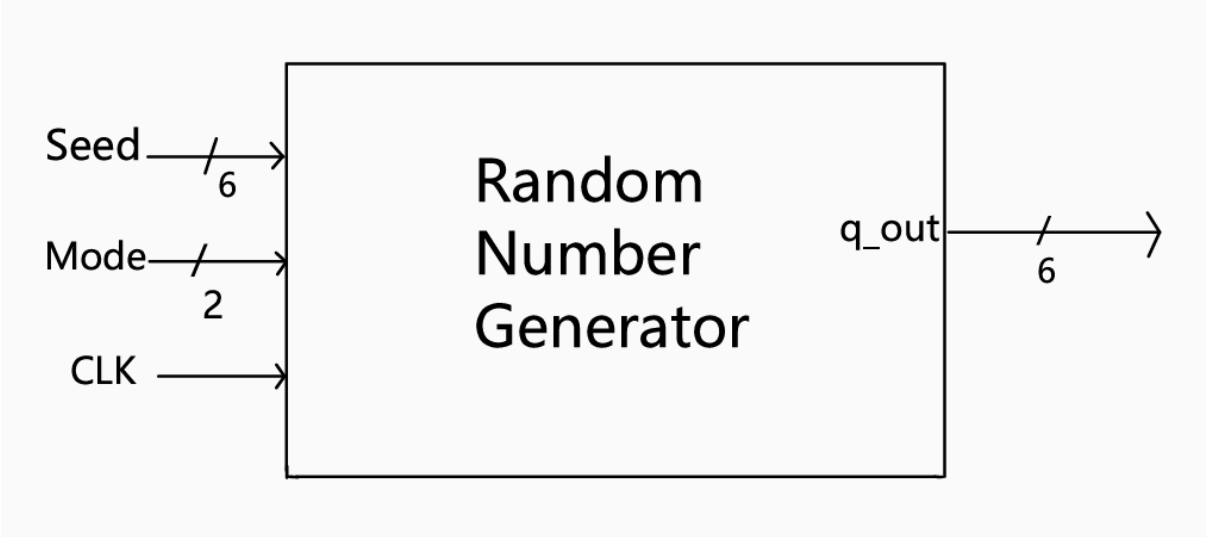
STACKS



The automated chain expansion that uncovers adjacent empty cells is a crucial element of Minesweeper. In standard software, this is executed using recursive function calls (e.g., Flood-Fill). However, VHDL cannot synthesize open-ended recursive loops because hardware layout paths must be fixed and finite at compile time.

To resolve this limitation, the design incorporates a dedicated **Last-In, First-Out (LIFO) hardware stack architecture**. The stack saves raw 6-bit grid indices of cells waiting for validation. This configuration converts a recursive software algorithm into a deterministic, clock-cycle-controlled iterative loop managed through clear hardware signals (PUSH, POP) and status flags (Empty_stack, Full_stack).

Random Number Generator



The random number generator operates based on the value of the **mode** input signal, which controls the behavior of the internal shift register.

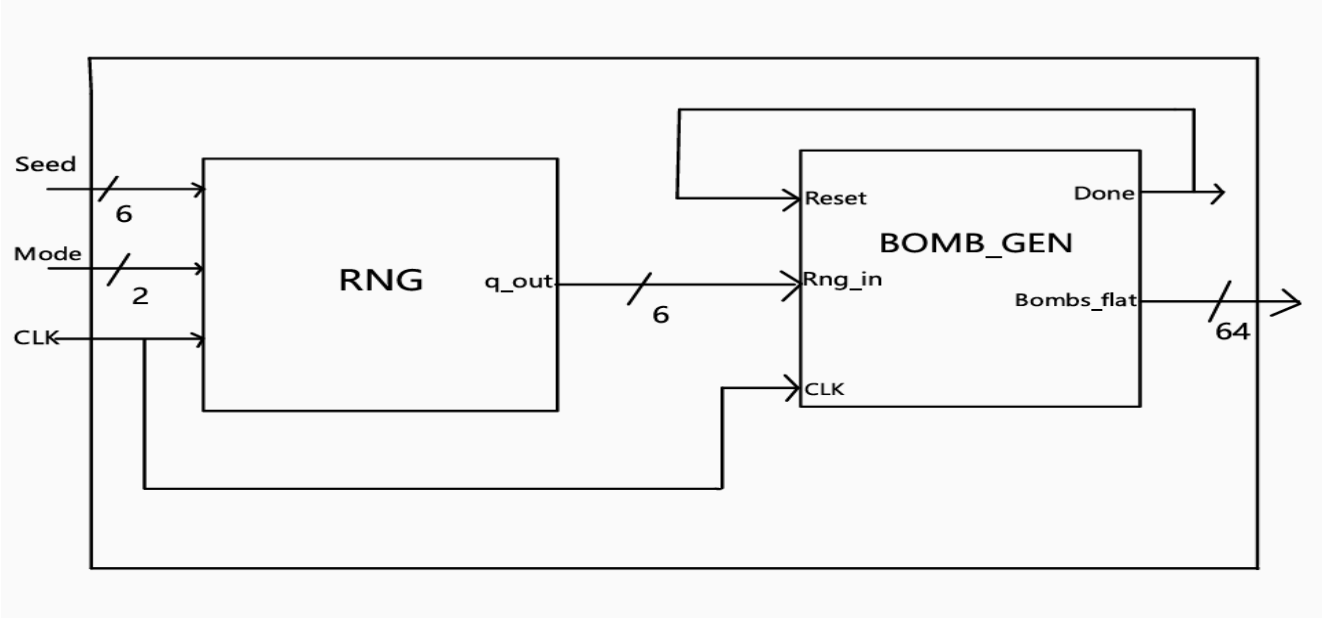
When mode = "11", the register is automatically initialized with the predefined binary value **"110101"**. This operation ensures a known starting state for the sequence generation.

When mode = "01", the register performs a right shift operation at each clock cycle. During this process, a new bit is inserted at the most significant position (MSB). This bit is computed as the exclusive OR (XOR) between the previous values of the most significant bit (MSB) and the least significant bit (LSB) of the register.

This mechanism allows the generator to produce a pseudo-random sequence of values based on feedback from its current state.

The generator is implemented as a 6-bit Linear Feedback Shift Register (LFSR).

Bomb generator top

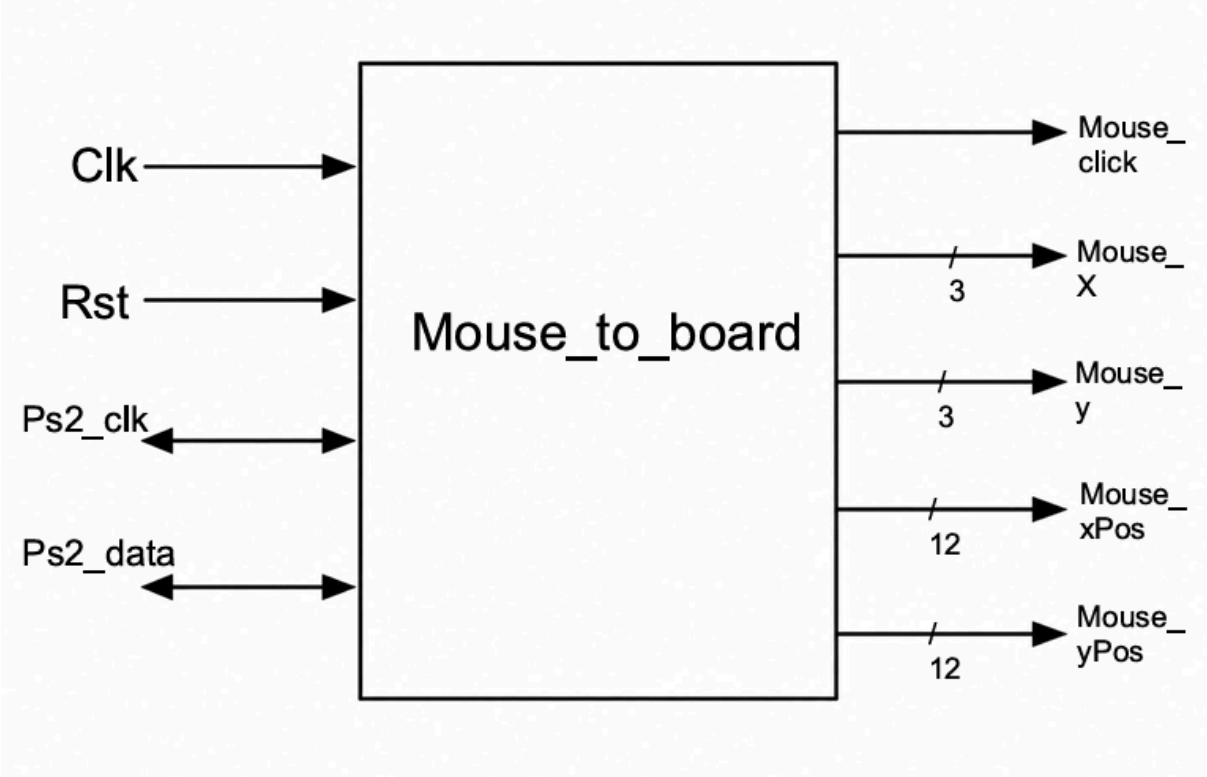


To bridge the raw pseudo-random numbers with the actual game mechanics, a dedicated **Bomb Generator Top (BOMB_GEN_TOP)** module is implemented. This component serves as the hardware manager that converts raw digital noise into a structured game grid.

The module captures the 6-bit pseudo-random values streamed out by the LFSR register. It passes these values through an internal decoder to map them to specific coordinates on the 8x8 playing field. Once the positions are processed, it compiles them into a single 64-bit flat array vector (Bomb_flat). Each bit position in this vector directly corresponds to one of the 64 unique tiles on the grid. A value of 1 represents a hidden mine, while a value of 0 indicates a safe cell.

In addition to placing the mines, the BOMB_GEN_TOP circuitry contains parallel arithmetic look-up logic. This logic scans the immediate 8-neighbor boundary of every single grid cell. It automatically calculates the total number of adjacent bombs for each coordinate. These computed density values are stored in the internal board map used by the game logic at startup. This enables instantaneous number lookups whenever a user selects a cell.

Mouse control



To provide a smooth user experience, the system uses a PS/2 mouse interface rather than simple onboard buttons or switches.

The Mouse_to_board receiver module handles the synchronous serial communication protocol. It monitors this protocol over the physical Ps2_Clk and Ps2_data lines.

The mouse hardware constantly transmits relative motion data packets. The receiver module aggregates these data packets into absolute coordinate positions (Mouse_xPos, Mouse_yPos). These absolute positions are mapped to the current VGA display area.

An integrated arithmetic scaling block processes these high-resolution coordinates. It scales them down into simple 3-bit grid cell indexes (Mouse_X/Mouse_Y). These resulting indexes range exactly from 0 to 7 to match the matrix boundary.

This conversion allows the finite state machine (FSM) to process clean user click events . The FSM can then confidently evaluate actions over the exact targeted coordinates.

5. CHAPTER 5 : FURTHER DEVELOPMENT

The current implementation of the Minesweeper game can be extended through several improvements aimed at increasing functionality and user flexibility. While the existing architecture sets up a stable and hardware-efficient foundation, expanding its scope allows the digital system to mimic professional, full-scale deployment benchmarks.

5.1 Dynamic and Expanded Grid Dimensions

One possible enhancement is the implementation of larger grid dimensions, such as 30×16 or 18×24, which would allow for more complex gameplay and increased difficulty.

- **Hardware Impact:** Scaling up the dimensions requires a major restructuring of the memory address bus. For instance, moving to a 30x16 grid expands the total cell count from 64 to 480. This requires upgrading the address bus from 6 bits to a minimum of 9 bits (2^9 = 512). An 18×24 grid introduces 432 cells, which comfortably fits within the same 9-bit address boundary.
- **Memory Management:** To accommodate this change without exhausting the FPGA's physical Block RAM (BRAM), the memory allocation strategy must be optimized. The storage requirements for both the Player View RAM and the underlying Board ROM will scale up linearly with the target grid layout.

5.2 Progressive Difficulty Levels

Another improvement consists in introducing multiple difficulty levels. These levels may differ in terms of grid size and bomb distribution, providing a progressive challenge for the player.

- **Preset Level Matrices:** The system can be designed to support distinct presets, seamlessly selected during the initial stage via hardware configurations:
    - *Beginner Level:* Retaining the baseline 8x8grid with a fixed 10-bomb layout.
    - *Intermediate Level:* Stepping up to a 16x16 layout hosting 40 mines.
    - *Expert Level:* Utilizing the maximum 30x16 or 18x24 layouts with a denser mine layout.
- **FSM Control Logic:** The Finite State Machine (FSM) will feature a dedicated difficulty setup state before moving into **ST_Initialize**. Depending on the player's choice, the clock-cycle runtime variables for the **BOMB_GEN_TOP** module will automatically adapt, adjusting both the density thresholds and the address ranges targeted by the VGA display rendering engine.

5.3 Hardware Flagging System

A core mechanic missing from the current prototype is the ability to place flags on suspected mine locations. Implementing flags prevents accidental clicks and allows players to track hidden threats.

- **Input Mapping:** The PS/2 mouse interface module will be upgraded to detect **right-click** events in addition to left-clicks.
- **Memory Architecture Adjustment:** The state tracking within the Player View RAM must be expanded. Currently, each cell uses 2 bits to store its visibility state (e.g., **00** for hidden, **01** for revealed). By expanding this memory word to 3 bits, a new state (**10**) can be introduced specifically to represent a "Flagged" cell.
- **Logic Constraints:** The Control Unit's FSM will be updated to check this state; if a cell is flagged, any subsequent left-click action on that coordinate will be safely ignored until the flag is removed by another right-click.

5.4 Custom Grid and Surface Configuration

Additionally, the system can be extended to support a custom configuration mode, where the user is allowed to define parameters such as grid dimensions and bomb density according to personal preferences.

- **Variable Control Registers:** By utilizing the onboard physical switches or an on-screen configuration menu, the user can manually input a custom grid width (up to a physical hardware cap), a custom grid height, and a target bomb count or density percentage.
- **Algorithmic Adaptability:** To support this degree of flexibility, the LFSR random number generator and the **BOMB_GEN_TOP** logic must be completely parameterized. Instead of relying on hardcoded modulo operations and static matrix boundaries, the execution unit will read user-defined surface constraints directly from dynamic registers to map out the boundaries and safely distribute the mines.

6. CHAPTER 6 : BIBLIOGRAPHY

- Wikipedia Minesweeper – *Minesweeper (video game)*
- Xilinx Inc. *VGA Controller Design Documentation*.
- Xilinx Inc. *Block RAM (BRAM) User Guide*
- VHDL for Engineers